Contents

CI/CD — Interview Questions & Answers **1**

CI/CD — Interview Questions & Answers

Q1: What is GitOps and how does it differ from traditional push-based CD? > **Traditional push-based CD**: The CI system (Jenkins, GitHub Actions) directly applies changes to the target environment after a build — it pushes Kubernetes manifests or runs Terraform. The CI system needs direct credentials to the cluster or cloud. > > **GitOps**: Git is the single source of truth for desired state. A controller running *inside* the cluster (ArgoCD, Flux) watches a Git repository and continuously reconciles actual state to match desired state. The cluster *pulls* from Git — no inbound credentials needed on the CI side. > > **Advantages of GitOps:** > - Full audit trail: every change to production is a Git commit with author, timestamp, PR link > - Drift detection: if someone `kubectl apply` directly, the controller detects and corrects it > - Rollback = `git revert` — deterministic and auditable > - No external write access to production cluster from CI systems

---

Q2: Explain DORA metrics. What do they measure and which matters most? > DORA (DevOps Research and Assessment) identified four metrics that predict software delivery performance: > > **Deployment Frequency**: How often code is deployed to production. Elite: multiple times per day. High: daily/weekly. Medium: weekly-monthly. Low: monthly or less. > > **Lead Time for Changes**: Time from code committed to running in production. Measures how quickly you can deliver value and fixes. Elite: < 1 hour. High: 1 day - 1 week. > > **Change Failure Rate**: Percentage of deployments that cause a production failure requiring hotfix, rollback, or remediation. Elite: 0-15%. Low: 46-60%. > > **Mean Time to Restore (MTTR)**: How long it takes to recover from a failure. Elite: < 1 hour. Low: 1 week to 1 month. > > **Which matters most**: MTTR combined with Change Failure Rate together — you want low failure rate AND fast recovery when failures happen. Deployment frequency is a leading indicator: the more you deploy small changes, the lower your failure rate and faster your recovery.

Q3: What is the difference between Continuous Integration, Continuous Delivery, and Continuous Deployment? > **Continuous Integration (CI)**: Every code change is automatically built and tested. Goal: detect integration failures early. Output: tested artifact (Docker image, JAR, binary). > > **Continuous Delivery (CD — delivery)**: Every successful build can be deployed to production at any time, but a human decides when to push the button. The pipeline prepares and validates the release. The deploy step requires manual approval. > > **Continuous Deployment (CD — deployment)**: Every successful build that passes all automated checks is automatically deployed to production with no human gate. Requires very high confidence in the test suite. > > Most mature engineering teams practice Continuous Delivery with automated canary gates that trigger rollback — functionally close to Continuous Deployment but with safety nets.

Q4: How do you implement zero-downtime deployments in Kubernetes? > Multiple layers required: > > **1. Deployment strategy**: `RollingUpdate` with `maxUnavailable: 0` — only scale up new pods before removing old ones. This ensures capacity is never reduced during a rollout. > > **2. Readiness probe**: The new pod only receives traffic after its readiness probe passes. A misconfigured or missing readiness probe will route traffic to unready pods. > > **3. PreStop lifecycle hook**: Add a `preStop` sleep of 5-15 seconds. This gives the load balancer time to remove the pod from rotation before SIGTERM is sent. Without this, in-flight requests can be dropped. > > **4. Graceful shutdown**: The application must handle SIGTERM and finish processing in-flight requests before exiting. `terminationGracePeriodSeconds` must be long enough. > > **5. PodDisruptionBudget**: Prevents all pods being removed simultaneously during node drains or voluntary disruptions. Set `minAvailable: 2` for a 3-replica deployment.

Q5: How do you handle database migrations in a CI/CD pipeline without downtime? > The **Expand-Contract pattern** (also called parallel change): > > **Phase 1 — Expand**: Add new schema elements (nullable columns, new tables) without removing anything. Deploy code that writes to both old and new schema. Old schema still works. > > **Phase 2 — Migrate**: Backfill existing data to new schema in small batches (avoid table locks). Tools: `pt-online-schema-change` or `gh-ost` for MySQL, or batched UPDATE loops for Postgres. > > **Phase 3 — Contract**: Once all data is migrated and code reads only from new schema, remove the old

columns/tables in a separate deployment. > > Each phase is a separate deployment, verified before the next. This is why you never do schema changes at the same time as application changes. > > **For non-relational changes**: Feature flags let you deploy code that uses a new data model without activating it — gradually migrate and enable per-user.

Q6: Walk through how you would design a secure secrets management strategy for a Jenkins pipeline. > **Avoid**: storing secrets as Jenkins Credentials with static values in the Jenkins config. These are encrypted with the Jenkins master key but don't rotate and are accessible to anyone with Jenkins admin access. > > **Better approach using HashiCorp Vault + JWT auth:** > 1. Jenkins job generates a short-lived JWT (role ID + secret ID via AppRole) > 2. Vault authenticates the JWT and issues a short-lived Vault token (TTL: 30 minutes) > 3. Pipeline retrieves only the secrets it needs for this specific job > 4. Secrets are injected as environment variables for the duration of the build only > 5. No secret survives the pipeline; nothing stored in Jenkins > > **AWS-native approach**: > - Jenkins agent runs with an IAM instance profile > - Agent assumes a per-pipeline IAM role (different permissions per job type) > - Retrieves secrets from AWS Secrets Manager at runtime > - No secret stored in Jenkins at all

---

Q7: What is a build artifact and why should you build once, deploy many times? > A **build artifact** is the immutable output of a CI build — a Docker image, a compiled binary, a JAR file, or a zip archive. It is stored in a registry or artifact repository with a unique version identifier (Git SHA, semantic version). > > **Build once, deploy many**: > - The same artifact that passed tests in staging is promoted to production — not rebuilt > - Rebuilding from source for production could produce a different binary (different dependency versions, different environment) > - The artifact is the unit of trust: security scanning runs against it once, signatures are applied once > - Faster promotions: no rebuild time, just pull and deploy > > **Implementation**: tag Docker images with Git SHA (`ghcr.io/org/app:sha-a1b2c3d`), push to registry after CI tests pass, then ArgoCD/Helm references that exact SHA for all subsequent environments.

Q8: How do you handle a broken main branch in a team of 50 engineers? > **Immediate actions:** > 1. Identify the breaking commit with `git bisect` or by examining recent CI failures > 2. Revert the specific commit: `git revert COMMIT_HASH` — creates a new commit, preserves history > 3. Push the revert — pipeline turns green > 4. Post in engineering Slack: "Main was broken by commit X, reverted in Y. Anyone with PRs based on those commits should rebase." > > **Communication**: Don't just silently revert — explain what happened so others don't build on broken code. > > **Longer fix**: Author rebases their branch on main (which now includes the revert), fixes the issue properly, re-opens PR. > > **Prevention**: > - Branch protection rules requiring PR reviews and passing CI before merging > - `CODEOWNERS` for critical files (requires specific team approval) > - Post-merge CI that re-runs tests after merge to catch integration issues > - Consider trunk-based development with feature flags to reduce long-lived branches

---

Q9: Explain the differences between GitHub Actions, GitLab CI, and Jenkins. When would you choose each? > **GitHub Actions**: > - Best for: teams on GitHub, open source projects, greenfield projects > - Pros: native GitHub integration, huge marketplace, OIDC to cloud providers, free for public repos > - Cons: less control over runners, GitHub outage affects CI > > **GitLab CI**: > - Best for: teams already on GitLab, enterprises wanting all-in-one (SCM + CI + registry + security) > - Pros: excellent built-in security scanning, environments/protected branches, self-hosted option > - Cons: GitLab-specific — harder to migrate away > > **Jenkins**: > - Best for: complex enterprise pipelines, full control requirements, existing Jenkins investment > - Pros: most flexible and extensible, huge plugin ecosystem, runs anywhere > - Cons: significant maintenance overhead (plugin updates, security patches, shared library management), operational toil > - In 2026: only choose Jenkins if migrating away from an existing Jenkins investment or if compliance requires fully on-prem

---

Q10: How do you measure and improve your CI/CD pipeline performance? > **Key metrics to track:** > - **Pipeline duration**: total time from push to deployed. Target: < 10 minutes for feedback > - **Stage duration breakdown**: which stage is the bottleneck? > - **Pipeline failure rate**: what % of runs fail and at which stage? > - **Queue wait time**: how long do jobs wait for a runner? > - **Flaky test rate**: tests that fail intermittently without code changes > > **Improvements by stage:** > > *Tests slow*: Parallelize test shards (`pytest-xdist`, matrix strategy), run only affected tests (path-based triggers), split unit from integration tests > > *Build slow*: Enable BuildKit layer caching, cache pip/npm in CI (`actions/cache`), multi-stage builds to skip

unnecessary layers > > *Image push slow*: Push to registry in same region as CI runners, use ECR/GHCR (not Docker Hub rate limited) > > *Deployment slow*: Reduce image size (distroless), pre-pull images to nodes, optimize readiness probe timing > > *Queue time high*: Add more runners, use auto-scaling runner controllers (ARC for GitHub Actions), right-size runner instances